

MonoGame으로 온라인 게임 클라이언트 만들기

C#과 MonoGame으로 배우는 멀티플레이 게임 클라이언트 개발

저자: 최흥배 · Claude AI

MonoGame 3.8

.NET 9

Visual Studio 2022

TCP/IP

목 차

1

MonoGame과 온라인 게임의 기초

1부: 기초 개념과 환경 구축

2

네트워크 통신의 기본

1부: 기초 개념과 환경 구축

3

바이너리 프로토콜 설계

2부: 네트워크 통신 구현

4

연결 관리와 메시지 처리

2부: 네트워크 통신 구현

5

MonoGame 프로젝트 시작

3부: MonoGame 게임 개발

6

플레이어 캐릭터 만들기

3부: MonoGame 게임 개발

7

플레이어 위치 동기화

4부: 온라인 기능 통합

8

멀티플레이 게임 완성

4부: 온라인 기능 통합

9

성능 최적화와 디버깅

5부: 심화 학습

10

다음 단계

5부: 심화 학습

1장: MonoGame과 온라인 게임의 기초

MonoGame이란?

- XNA Framework 기반 오픈소스 크로스플랫폼 게임 프레임워크
- C# / .NET으로 개발, 낮은 추상화 → 성능과 제어권 보장
- Windows · macOS · Linux · iOS · Android 지원
- 인디 게임에 적합, Terraria · Celeste 등이 MonoGame으로 제작됨

온라인 게임 클라이언트 구조

- 클라이언트: 사용자 입력 처리, 렌더링, 서버 통신
- 서버: 게임 상태 관리, 클라이언트 동기화
- 권한 서버(Authoritative Server) 모델 → 치트 방지
- 패킷 기반 데이터 교환 (TCP/UDP)

개발 환경 구축

- Visual Studio 2022 Community 이상
- .NET 9 SDK
- MonoGame 3.8 NuGet 패키지
- `dotnet new mgdesktopgl` 프로젝트 생성

핵심 개념: 게임 루프 (Game Loop)

- Initialize: 게임 초기화 (리소스 로드, 변수 설정)
- LoadContent: 텍스처·사운드 등 콘텐츠 로드
- Update(GameTime): 게임 로직 처리, 입력 처리, 네트워크 수신
- Draw(GameTime): SpriteBatch로 화면 렌더링

TCP vs UDP

- TCP: 연결 지향, 순서·무결성 보장, 게임 로그인·채팅에 사용
- UDP: 비연결, 빠른 전송, 위치 동기화에 활용
- 소켓(Socket): 네트워크 통신의 끝점(endpoint)
- 포트: 동일 호스트에서 여러 프로세스 구분

클라이언트-서버 아키텍처

- 클라이언트: 입력 전송, 렌더링, UX 처리
- 서버: 게임 상태 권한(Authority) 보유
- 3-way 핸드셰이크로 TCP 연결 수립
- ConnectAsync로 비동기 연결 (타임아웃 5초)

바이너리 직렬화 원리

- JSON·XML 대비 낮은 오버헤드 → 게임에 최적
- BinaryPrimitives: 정수·부동소수 직렬화
- Little-Endian 바이트 순서 일관 사용
- 패킷 = 헤더(타입+길이) + 페이로드

TCP/IP 4계층 (게임 통신 관점)

응용 계층

게임 패킷 프로토콜 (패킷 타입, 직렬화)

전송 계층

TCP (신뢰성) / UDP (속도)

인터넷 계층

IP 라우팅, 주소 지정

네트워크 인터페이스

물리적 데이터 전송 (이더넷 등)

패킷 구조

PacketType (1 byte)

패킷 종류 식별자
(Login=1, Position=2, Chat=3...)

Length (2 bytes)

페이로드 전체 길이
BinaryPrimitives.WriteUInt16LittleEndian

Payload (N bytes)

실제 데이터
(플레이어ID, 좌표, 메시지 등)총 헤더: 3 bytes / LoginPacket: ~12B / PositionUpdate: 15B / ChatPacket: 가
변

PacketType 상수 정의

- Login = 1, LoginResponse = 2
- PositionUpdate = 3, ChatMessage = 4
- PlayerJoin = 5, PlayerLeave = 6
- HeartBeat = 10

BinaryPrimitives 직렬화

- WriteInt32LittleEndian / ReadInt32LittleEndian
- WriteSingleLittleEndian → float 좌표 기록
- Span<byte> 활용으로 힙 할당 최소화
- PacketBuffer로 수신 버퍼 누적 관리

```
// 패킷 송신 예시
var buf = new byte[3 + payload.Length];
buf[0] = (byte)PacketType.PositionUpdate;
BinaryPrimitives.WriteUInt16LittleEndian(
    buf.AsSpan(1), (ushort)payload.Length);
payload.CopyTo(buf.AsSpan(3));
await _stream.WriteAsync(buf);
```

서버 연결 관리 (NetworkClient)

- ConnectAsync: CancellationToken 5초 타임아웃
- SendPacketAsync: 직렬화 후 스트림 전송
- StartReceivingAsync: 백그라운드 수신 루프
- 연결 해제·재연결·에러 핸들링 포함

비동기 패킷 수신

- NetworkStream.ReadAsync로 헤더 3바이트 수신
- 페이로드 길이 파싱 후 나머지 수신
- PacketReceiver 클래스로 수신 로직 캡슐화
- 멀티스레드 환경에서 안전한 버퍼 처리

메시지 처리 패턴

- ConcurrentQueue<Packet>으로 스레드 안전 큐
- MessageProcessor.RegisterHandler(type, handler)
- Update() 루프에서 큐 Dequeue & 디스패치
- Game Thread에서만 상태 변경 → 동기화 문제 해소

메시지 흐름도

서버

→ NetworkStream
수신→ PacketReceiver
파싱→ ConcurrentQueue
입큐→ Update()
디스패치→ 게임 상태
업데이트

프로젝트 생성

- `dotnet new mgdesktopgl -n MyGame`
- Content/ 폴더: 리소스 파일 관리
- Game1.cs: 기본 게임 클래스 (Game 상속)
- MGCB Editor로 콘텐츠 빌드 파이프라인 관리

기본 게임 루프 이해

- `Initialize()`: 변수 초기화, 네트워크 연결 시작
- `LoadContent()`: Texture2D·SpriteFont 로드
- `Update(GameTime)`: 입력·네트워크·로직 처리
- `Draw(GameTime)`: `SpriteBatch.Begin/Draw/End`

GameManager 분리 패턴

- Game1 클래스를 단순 진입점으로 유지
- GameManager가 플레이어·네트워크·UI 관리
- SceneManager로 화면 전환 추상화 가능
- 관심사 분리 → 유지보수성 향상

```
// Update: 게임 로직 처리
void Update(GameTime gt) {
    float dt = (float)gt.ElapsedGameTime
                .TotalSeconds;
    _inputManager.Update();
    _networkClient.ProcessMessages();
    _player.Update(dt);
}
// Draw: 렌더링
void Draw(GameTime gt) {
    _spriteBatch.Begin();
    _player.Draw(_spriteBatch);
    _spriteBatch.End();
}
```

렌더링 핵심 메서드

- `SpriteBatch.Begin()/End()` — 렌더 배치 시작/종료
- `Draw(texture, position, color)` — 기본 스프라이트 그리기
- `DrawString(font, text, pos, color)` — UI 텍스트 출력
- `MathHelper.Clamp()` — 화면 경계 제한
- `IsFixedTimeStep = true` — 고정 프레임레이트 (60fps)

6 6장: 간단한 플레이어 캐릭터 만들기

Sprite & Texture2D

- Texture2D: GPU에 업로드된 이미지 리소스
- SpriteGenerator: Content 없이 동적 텍스처 생성
- Content.Load<Texture2D>("player") — 파일 로드
- Rectangle(x,y,w,h)로 스프라이트 시트 잘라내기

InputManager

- Keyboard.GetState()로 현재 키 상태 조회
- IsKeyDown: 키 누르는 중 / IsKeyUp: 키 떴을 상태
- IsKeyPressed: 이번 프레임에 처음 눌림 (엣지 감지)
- IsKeyReleased: 이번 프레임에 땀 (엣지 감지)

Player 클래스

- Position(Vector2), Velocity(Vector2), Speed
- PlayerState enum: Idle/Moving/Attacking
- 대각선 이동 시 Velocity.Normalize() 필수
- Animation 상태에 따른 스프라이트 변경

```
// 대각선 이동 정규화 처리
var velocity = Vector2.Zero;
if (_input.IsKeyDown(Keys.W)) velocity.Y -= 1;
if (_input.IsKeyDown(Keys.S)) velocity.Y += 1;
if (_input.IsKeyDown(Keys.A)) velocity.X -= 1;
if (_input.IsKeyDown(Keys.D)) velocity.X += 1;
if (velocity.LengthSquared() > 0)
    velocity.Normalize(); // 대각선도 동일 속도 유지
Position += velocity * Speed * deltaTime;
Position = Vector2.Clamp(Position, Vector2.Zero, _bounds);
```

PlayerState & 애니메이션 흐름

- Idle → Moving: 방향키 입력 시
- Moving → Idle: 모든 방향키 해제 시
- Moving → Attacking: 공격키 입력 시
- 각 State에 따른 스프라이트 프레임 선택
- animTimer로 프레임 교체 타이밍 관리

PositionUpdate 패킷 (15 bytes)

- 1B: PacketType.PositionUpdate
- 2B: 페이로드 길이
- 4B: Player ID (int32)
- 4B: X 좌표 (float)
- 4B: Y 좌표 (float)
- 1B: Direction (byte)

PositionSynchronizer

- 0.1초(100ms) 간격으로 위치 패킷 전송
- 임계치(Threshold) 이상 이동 시에만 전송 (최적화)
- syncTimer += deltaTime으로 전송 타이밍 관리
- 마지막 전송 위치와 현재 위치 비교

RemotePlayerManager

- Dictionary<int, RemotePlayer>로 원격 플레이어 관리
- UpdatePlayer(id, x, y, dir): 수신 위치 업데이트
- 마지막 수신 시간 기록 → 타임아웃 시 제거
- 각 RemotePlayer는 선형 보간(Lerp)으로 부드럽게 이동

위치 동기화 흐름

내 플레이어
이동

→ Synchronizer
0.1초 체크

→ 서버로
패킷 전송

→ 서버
브로드캐스트

→ 다른 클라
위치 수신

→ Lerp 보간
부드러운 이동

채팅 기능

- ChatPacket: BinaryWriter로 가변 길이 직렬화
- BinaryReader.ReadString()으로 역직렬화
- TextBox UI로 입력 처리
- 채팅 메시지 리스트 화면에 오버레이 표시

GameState 관리 (enum)

- Connecting: 서버 연결 대기
- Connected: 연결 완료, 로그인 전
- InGame: 게임 플레이 중
- Disconnected: 연결 끊김
- Error: 오류 발생 상태

CompleteMultiplayerGame

- PlayerJoinPacket / PlayerLeavePacket 처리
- NetworkHandler로 패킷 타입별 분기
- HUD: 접속 플레이어 수, 레이턴시 표시
- 로컬·원격 플레이어 통합 렌더링

완성된 멀티플레이 게임 구조

InputManager
(키 입력)Player
(로컬)RemotePlayer
ManagerNetworkClient
(TCP 연결)ChatSystem
(채팅)HUD/UI
(오버레이)

↑ 모두 CompleteMultiplayerGame 클래스에서 통합 관리

네트워크 트래픽 최소화

- 델타 압축: 변경된 좌표만 전송 (임계치 비교)
- 메시지 배칭: 큐에 모았다가 일정 주기로 일괄 전송
- 상태 압축: float(4B) → short(2B), 방향 → 1byte
- 전송 빈도 조절: 위치 100ms, 채팅은 즉시

프레임 동기화

- IsFixedTimeStep = true (기본 60fps)
- TargetElapsedTime으로 목표 프레임 설정
- NetworkTimingSynchronization: 서버 시간 동기
- 클라이언트 사이드 예측(Prediction)으로 지연 은폐
- 롤백(Rollback) 메커니즘 기초

디버깅 도구

- NetworkPacketAnalyzer: 패킷 로그 기록·분석
- PerformanceMonitor: FPS·레이턴시·패킷수 추적
- GameLogger: 파일 기반 로그, 레벨별 필터링
- Wireshark로 실제 네트워크 트래픽 분석 가능

주요 최적화 기법 비교

델타 압축

이전: 매 프레임 x,y 전송 (8B)

이후: 변화량만 전송 (2~4B)

~50% 절감

메시지 배칭

이전: 패킷마다 즉시 전송

이후: 100ms마다 묶어서 전송

TCP 헤더 오버헤드 감소

상태 압축

이전: float 좌표 (4B×2=8B)

이후: short 좌표 (2B×2=4B)

50% 크기 감소

고급 네트워킹 라이브러리

- Photon PUN: 클라우드 기반 멀티플레이 서비스
- LiteNetLib: 경량 UDP 라이브러리 (C#)
- Mirror / Netcode for GameObjects (Unity)
- ENet: 신뢰성 있는 UDP, 저지연

게임 확장 방향

- Item / Inventory / Combat System
- Quest / QuestManager 시스템
- QuadTree로 공간 분할 → 광역 쿼리 최적화
- Animator / ParticleEmitter / SoundManager
- PlayerSaveData / GameSaveManager (세이브 시스템)

NetworkObject 패턴

- INetworkObject 인터페이스로 추상화
- NetworkObjectManager로 통합 관리
- NetworkBehaviour + RPC 패턴
- 속성 자동 동기화 (SyncVar 개념)

프로젝트 완성 체크리스트

핵심 게임플레이

- 플레이어 이동
- 멀티플레이 동기화
- 채팅 시스템
- 입장/퇴장 처리

기술 요소

- 네트워크 에러 처리
- 재연결 로직
- 패킷 암호화 고려
- 서버 인증

UX / 출시

- 로딩 화면
- 설정 저장
- 패키징 (Windows EXE)
- 성능 프로파일링

학습 여정을 마치며

이 책에서 배운 것

- 1부 기초: MonoGame 구조와 .NET 네트워크 API 이해
- 2부 구현: TCP 바이너리 프로토콜 설계 및 패킷 핸들러 구현
- 3부 게임: MonoGame 게임 루프, 스프라이트, 플레이어 구현
- 4부 통합: 위치 동기화, 채팅, 완성된 멀티플레이 게임
- 5부 심화: 성능 최적화, 디버깅, 고급 패턴 및 확장 방향

핵심 기술 스택

- C# / .NET 9
- MonoGame 3.8
- TCP/IP 소켓
- BinaryPrimitives

다음 도전 과제

- 서버 사이드 구현
- Photon 클라우드 연동
- Unity로 확장
- 상용 게임 출시